

Intent2Tc: Automated Intent-to-Traffic Control Translation with Language Models

Andrea Masini, Sudipta Acharya, Paolo Bellavista, Luca Foschini, Burak Kantarci

Abstract—Automated and highly usable Quality-of-Service (QoS) enforcement requires translating high-level service intents into deployable traffic-management policies. Although intent-based networking (IBN) has simplified policy specification, bridging the gap between business-level intents and executable network configurations remains complex, error-prone, and difficult to automate. This paper presents Intent2Tc, a closed-loop language-model-driven framework that translates business-level traffic-shaping intents into declarative sub-intents and subsequently into validated, executable Linux traffic control (tc) configurations. The framework integrates an Active Queue Management (AQM)-based digital twin (DT) semantic model, automated metadata extraction, critique-driven refinement, and Retrieval-Augmented Generation (RAG)-based knowledge reuse to improve semantic consistency and configuration reliability. We evaluate multiple open-source large language models (LLMs) and small language models (SLMs), together with Claude Sonnet-4.6, on 100 Request for Comments (RFC) 9315-compliant traffic-shaping intents. Across both translation stages, Intent2Tc achieves high semantic fidelity, configuration accuracy, and deployment readiness, with Claude Sonnet-4.6 reaching 0.98 semantic similarity, 1.0 semantic unit coverage, and 0.045 normalized edit distance. Furthermore, RAG reduces token consumption and inference latency while enabling compact models such as Phi-4-mini to approach the performance of substantially larger models. Linux tc serves as the target configuration platform, demonstrating the practical applicability of the proposed framework.

Index Terms—IBN, Linux tc, language models, RAG.

I. INTRODUCTION

Interest in IBN frameworks has grown significantly in recent years [1], [2], driven by the need to simplify traffic management and enable autonomous actuation, supervision, and adaptation across various scenarios [3]. Research shows that most existing IBN work focuses on the underlying infrastructure layer [4], while the intent-processing pipeline remains dependent on slow, repetitive, and error-prone manual creation of network-shaping rules. More recently, language models (LMs) have emerged as a pivotal approach for intent translation, mediating between high-level human articulations and the underlying network configuration primitives [5], [6]. Despite notable advances in the validation and orchestration stages [6], the terminal translation phase is largely left to manual intervention [7]. Intent2Tc addresses this gap as a natural-language-to-tc translator focused on the acquisition-and-translation stage of the IBN lifecycle.

Andrea Masini, Sudipta Acharya and Burak Kantarci are with the University of Ottawa, Ottawa, ON, Canada. Emails: {amasi092,sacharya2,burak.kantarci}@uottawa.ca
Andrea Masini, Paolo Bellavista and Luca Foschini are with the University of Bologna, Italy. Emails: andrea.masini6@studio.unibo.it & {paolo.bellavista,luca.foschini}@unibo.it
This work was performed while Andrea Masini was affiliated with the University of Ottawa, working in B. Kantarci’s research lab.

Recently, the authors of [8] introduced the first queueing-theory-based end-to-end LM-driven intent-to-tc translation pipeline, but it relied on static keyword matching and a single-pass critique, which limited recovery from mismatches and learning from validated outputs. To overcome these limitations, we propose a *closed-loop* framework with dynamic metadata extraction, multi-stage critique, and RAG-based knowledge reuse to improve accuracy, robustness, and adaptability.

Our framework follows a three-phase workflow: (1) an AQM-based DT provides a *semantic model* of the simulated priority-queueing network; (2) an LM extracts metadata (aided by a confidence-based common-patterns lookup table (LUT)), decomposes the intent into declarative *sub-intents*, and generates Linux tc rules, while a Critique Module (CM) corrects hallucinations, duplicates, and omissions after each stage; (3) the RAG database (RAG DB) is updated with the corrected intent, sub-intents, and tc rules, and indexed by traffic profile and time bounds. The key contributions of this work are as follows:

- A scalable DT environment implementing priority-class queueing with AQM, enabling semantic-model generation for diverse traffic-shaping scenarios.
- A RAG DB of validated intents, sub-intents, and tc rules to support continual improvement in intent translation.
- A closed-loop LM-driven framework for translating traffic-shaping intents into deployable tc rules through DT-based semantic modeling, dynamic traffic-profile matching, confidence-based metadata extraction, critique-driven correction, and RAG-assisted knowledge reuse.

To evaluate end-to-end intent-to-tc translation, we transform a publicly available business-intent dataset [9] into 100 RFC 9315-compliant traffic-shaping intents [2], preserving their real-world context. We evaluate multiple open-source LLMs and SLMs available on Hugging Face [10] with and without RAG augmentation, including Hermes-2-Pro (8B), Qwen-3 (4B/8B), Phi-4-mini (3.8B), Llama-3.1 (8B), Qwen3.5-Claude-Opus4.6-Distill (9B), Gemma-4-E4B (8B), and IBM Granite-4.1 (8B), using Claude Sonnet-4.6 as a closed-source baseline. The reported results show highly accurate traffic-profile and time-bound extraction, while RAG reduces token usage and inference latency. Claude Sonnet-4.6 achieves 0.98 semantic similarity, 1.0 semantic unit coverage, and 0.045 normalized edit distance.

The remainder of this paper is structured as follows. Section II reviews related work and identifies the gap in closed-loop intent-to-tc translation. Section III presents the proposed framework, including semantic modeling, metadata extraction, critique-driven correction, and RAG-based knowledge reuse. Section IV describes the dataset, evaluation metrics, case

study, and results. Section V concludes the paper and outlines future work.

II. RELATED WORK

Linux `tc` is a widely used but complex tool for enforcing QoS in cloud-native environments such as Kubernetes. It supports classful and classless schedulers, packet filtering, priority-based scheduling, and traffic-shaping mechanisms, such as AQM. However, its strict syntax and kernel-level complexity make manual rule generation difficult and error-prone [11]. Recent work shows that LMs can help parse natural-language intents [12] and automate parts of the intent lifecycle, but the final step of turning validated intents into platform-specific configurations remains challenging per RFC 9315 [2]. Frameworks such as INTA [13] use intents as an intermediate representation to bridge heterogeneous configuration models, while NetConfEval [7] shows that LLMs can translate high-level policies into formal specifications for software-defined networking (SDN) controllers; however, support for the more complex syntax of Linux `tc` remains limited. NetIntent [14] automates the intent lifecycle using LLMs for conflict detection and resolution. However, most LLMs remain prone to hallucinations and can produce technically plausible but incorrect configurations.

Recently, the authors of [8] attempted to close this gap, proposing an LM-based pipeline for intent-to-QoS rule generation. In their work, they assess the performance of different prompting strategies in sub-intent generation and `tc` rule translation, identifying few-shot prompting as the best-performing approach. They specifically state the need for an iterative learning loop, describing it as a key missing component of their infrastructure.

Overall, existing studies improve intent interpretation, validation, and configuration generation, but they mostly rely on a single-pass translation process. This leaves limited room for knowledge reuse, iterative refinement, or learning from previous translations, motivating a closed-loop intent-to-`tc` framework with semantic modeling, critique-based validation, and retrieval-driven knowledge accumulation.

III. METHODOLOGY

Fig. 1 illustrates the proposed architecture and intent translation loop, while Algorithm 1 summarizes the overall workflow.

a) *Phase 1: AQM DT Semantic Model Setup*: The initial phase builds a `SimPy`-based network DT using a finite-capacity, non-preemptive queueing model with priority classes calibrated offline using queueing theory. It uses Controlled Delay (CoDel)-based AQM to favor dropping low-priority packets under overload, while remaining extensible to other mechanisms, such as Random Early Detection (RED). Traffic arrivals follow a Poisson process, and service rates are tuned to meet target utilization levels. During simulation, packets are scheduled according to priority and managed according to queue policies under overload conditions. The resulting performance metrics are organized into a semantic model that links traffic objectives to feasible system behavior and supports subsequent phases of intent interpretation and configuration generation.

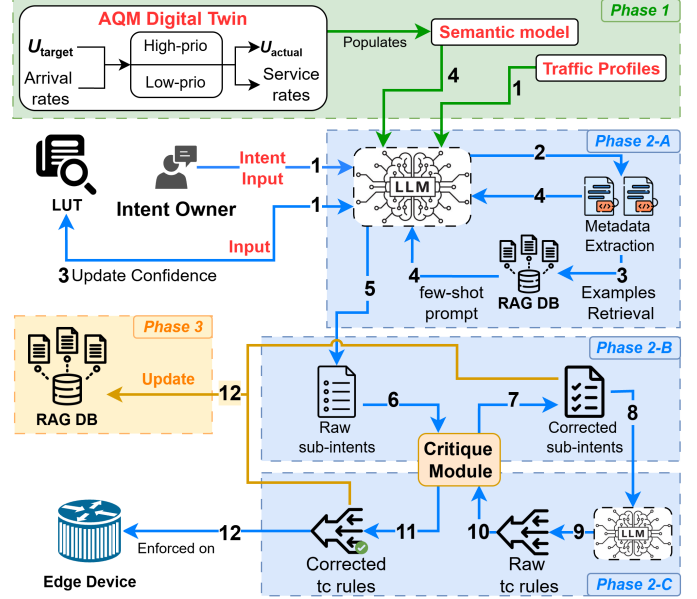


Fig. 1. Proposed intent-to-`tc` pipeline: numbered arrows show execution order; unnumbered arrows show independent contextual flows.

b) *Phase 2: Metadata Extraction, Intent Subdivision, and Rule Generation*: In the second phase, an LM-driven pipeline transforms high-level traffic-shaping intents into deployable Linux `tc` rules using four sources of context: the user intent, a confidence-based LUT, the semantic model from Phase 1, and a traffic-profile taxonomy.

Phase 2-A – Helper Metadata Extraction: The LM extracts auxiliary metadata from the user intent, including the traffic profile, priority level, and time bounds, to guide subsequent translation stages. The confidence-based LUT maps recurring natural-language expressions to priority and temporal constraints, providing additional contextual guidance.

When RAG is enabled, the system retrieves the two most relevant examples from the RAG DB that match the extracted traffic profile and injects them into the sub-intent and rule-generation prompts. Otherwise, it falls back to a generic few-shot prompting strategy. Each RAG DB entry stores a *slot* (`traffic_profile::time_bounds`), the original intent, corrected sub-intents, corrected `tc` rules, and the associated Token-F1 score.

Phase 2-B1 – Generation of Declarative Sub-Intents: From the high-level intent, the LM generates declarative sub-intents expressing network requirements and queueing policies. Collectively, they form a fundamental intermediate interface bridging the general intent and low-level configuration. The prompt draws from three main context sources: extracted metadata, the semantic model, and either dynamic RAG-retrieved or static few-shot examples. The generated sub-intents capture performance constraints, such as delay and packet-drop thresholds; traffic prioritization requirements, including the appropriate priority class; AQM-related thresholds for congestion management; and temporal constraints governing policy enforcement.

Phase 2-B2 – Sub-Intent Correction: The generated sub-intents are subsequently validated by a deterministic, template-based CM. The CM identifies and removes invalid or policy-

inconsistent sub-intents, inserts any missing requirements implied by the original intent and semantic model, and merges semantically redundant entries. The result is a complete, consistent, and logically coherent set of corrected sub-intents for downstream rule generation.

Phase 2-C1 – Low-Level Linux tc Rule Generation: In the final stage of Phase 2, the LM translates the corrected sub-intents into executable Linux tc rules. The generation process is guided by traffic-profile attributes (e.g., Internet Protocol (IP) addresses, ports, and protocols), the semantic model, and either RAG-retrieved or few-shot examples to ensure that the resulting configurations remain consistent with both the intent requirements and the underlying traffic characteristics.

Phase 2-C2 – tc Rule Correction: The generated tc rules are then validated and refined by the CM to produce a consistent, syntactically correct, and deployment-ready configuration set. The CM enforces key tc -specific constraints, including the Hierarchical Token Bucket (HTB) class hierarchy (e.g., `classid 1:10/1:11`), priority mapping (e.g., `prio 0/prio 2`), `fq_codel` AQM parameters, flower-based packet filtering, and strict syntactic ordering, since swapping two operators in a tc rule can trigger cascading failures. All modifications are logged to provide an auditable trail of the translation and correction process.

Algorithm 1: Intent-to- tc configuration: semantic modeling, critique correction, and RAG optimization

```

Data: Natural-language intents  $I_{\text{list}}$ , critique templates (sub-intents  $R_d$ ,  $\text{tc}$  rules  $R_c$ ), and RAG flag  $\text{USE\_RAG}$ 
Result: Corrected sub-intents  $\hat{I}$  and valid  $\text{tc}$  rules  $T_i$ 
1 /* Phase 1: Semantic Modeling */
2 Set  $(\lambda_{\text{high}}, \lambda_{\text{low}}, \mu_{\text{high}}, \mu_{\text{low}})$ ;  $S \leftarrow$  SimPy priority queue with CoDel;
3 /* Phase 2: Main Loop */
4 Load common patterns  $CP$ , traffic profiles  $TP_{\text{list}}$ , and few-shot examples  $E$ ;
5 foreach  $i \in I_{\text{list}}$  do
6   /* Phase 2-A: Metadata Extraction */
7    $M_d \leftarrow \text{LM\_extract}(\text{build\_ext\_prompt}(i, CP, TP_{\text{list}}))$ ;
8    $(Prof, Prio, Time, Synt_{id}, Synt_{con}, Time_{id}, Time_{con}) \leftarrow M_d$ ;
9    $\text{update\_CP}(Synt_{id}, Synt_{con}, Time_{id}, Time_{con})$ ;
10   $P \leftarrow \text{profile\_filter}(Prof)$ ;
11   $(R_d, R_c) \leftarrow \text{build\_critique}(M_d, P)$ ;
12  if  $\text{USE\_RAG}$  then
13     $E \leftarrow \text{rag\_retrieve}(P :: Time)$ 
14  end
15  /* Phase 2-B: Sub-intent Generation */
16   $\hat{I} \leftarrow \text{fix\_subs}(\text{LM\_subintents}(\text{build\_sub\_prompt}(i, S, M_d, E)), R_d)$ ;
17  /* Phase 2-C: Linux  $\text{tc}$  Rules Generation */
18   $T_i \leftarrow \text{fix\_tc}(\text{LM\_tc}(\text{build\_cfg\_prompt}(\hat{I}, S, P, E)), R_c)$ ;
19  /* Phase 3: RAG DB Update */
20  if  $\text{USE\_RAG}$  then
21    RAG DB  $\leftarrow \text{rag\_update}(P :: Time, i, \hat{I}, T_i, \text{compute\_TokenF1})$ 
22  end
23  /* Output */
24  Save  $\hat{I}$  and  $T_i$  as JavaScript Object Notation (JSON); log corrections
    and motivations;
25 end

```

c) **Phase 3 – Updating the RAG DB:** Finally, when RAG is enabled, the system evaluates the corrected translation for possible inclusion in the RAG DB. A `Token-F1` score is computed, and the new entry is inserted only if its score exceeds that of at least one existing record. `Token-F1` was selected as a lightweight proxy for pre-deployment comparison, capturing token-level overlap between generated and ground-truth outputs. The RAG DB is initially empty and is progressively populated with validated intent-sub-intent- tc rule mappings as the system processes successive intents.

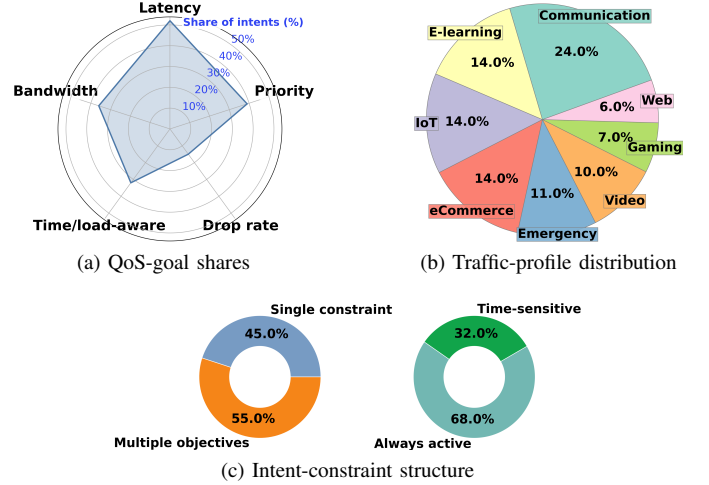


Fig. 2. 100-intent dataset statistics

d) **Output:** The final output of the framework is a validated and deployment-ready set of Linux tc rules for traffic-shaping enforcement.

IV. PERFORMANCE EVALUATION

A. Dataset and Evaluation Metrics

As outlined in Section I, the 100-intent benchmark was derived from the business-intent dataset of [9]. We selected 100 intents spanning diverse traffic profiles, QoS objectives, and temporal requirements, and transformed them into RFC 9315-compliant traffic-shaping intents [2], preserving the original operational objectives while abstracting implementation-specific parameters. Ground-truth sub-intents and Linux tc configurations were then derived and validated by experienced network administrators and traffic engineers, providing a deployment-ready reference for evaluating LM-generated outputs.

Fig. 2 summarizes the composition of the 100-intent dataset. As shown in Fig. 2a, the dataset covers diverse QoS objectives, including latency, bandwidth, priority, time/load-aware control, and packet-drop constraints, with stronger emphasis on latency and priority-related goals due to their central role in practical traffic shaping. Fig. 2b shows that the traffic taxonomy spans communication services, the Internet of Things (IoT), e-learning, emergency applications, e-commerce, video streaming, gaming, and Web traffic, ensuring coverage of heterogeneous application scenarios. Finally, Fig. 2c illustrates the generality of the considered intents, which include both single-constraint and multi-objective requirements, as well as always-active and time-sensitive policies.

The lack of established benchmarks for intent-to- tc translation motivated our dataset construction and necessitates evaluation across semantic fidelity, configuration accuracy, and deployment readiness. For sub-intent generation, we use Sentence Bidirectional Encoder Representations from Transformers (SBERT) cosine similarity [15], Recall-Oriented Understudy for Gisting Evaluation–Longest Common Subsequence (ROUGE-L) F1 [16], Token-F1 [17], and Metric for Evaluation of Translation with Explicit Ordering (METEOR) [18]. For tc rule generation, we evaluate semantic unit coverage [19],

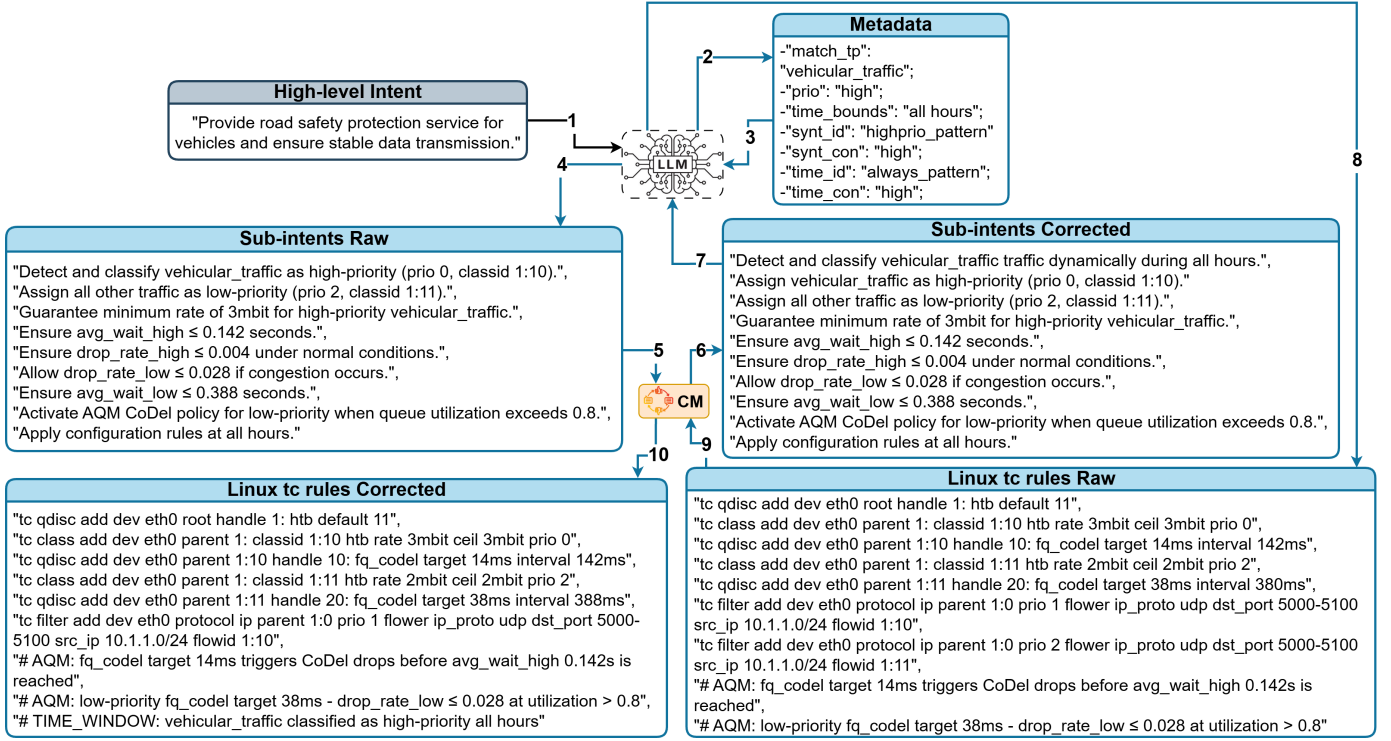


Fig. 3. Case study: high-level intent to critique-corrected, deployable Linux `tc` configurations; numbered arrows show processing order.

Note: HTB classes 1:10 and 1:11 use the `fq_codel` AQM queueing discipline (`qdisc`); the `target/interval` parameters enable congestion-driven early drops and Explicit Congestion Notification (ECN). The `flower` classifier matches protocol, ports, and IP, steering traffic to high-priority 1:10 with hardware offload and no `u32` masks. Future dynamic deep packet inspection (DPI) extensions use `fwmark` for non-port flows.

Token-F1, and normalized edit distance (NED) [20]. To assess deployment readiness beyond these metrics, we incorporate the Format, Explainability, Accuracy, Cost, and Inference Time (FEACI) framework [6], covering syntactic validity, technical correctness, operational feasibility, and deployment efficiency. Since some FEACI scores require human assessment, we derive automated counterparts (Table I) per the authors' definitions.

B. Case Study: Stabilize Vehicular Traffic Data Transmission

To illustrate the critique-and-correction workflow, Fig. 3 presents a real-time vehicle-to-everything (V2X) intent. Prior to deployment, the CM performs the following corrections to the generated sub-intents and `tc` rules. In **Phase 2-B**, the CM splits a malformed sub-intent into separate profile/time-bound detection and priority-assignment sub-intents. In **Phase 2-C**, the CM removes a hallucinated filtering rule for the low-priority HTB class 1:11 and inserts the missing `TIME_WINDOW` application annotation into the rule set.

C. Analysis of Results

The evaluations are conducted using static few-shot and dynamic RAG-based prompt construction, focusing on sub-intent generation from high-level intents and translation into Linux `tc` rules.

1) *Dynamic Metadata Extraction Analysis:* We compare the proposed LM-based dynamic metadata extraction with the static keyword-matching logic of [8]. With Claude Sonnet-4.6

as the metadata extraction model, the dynamic approach improves traffic-profile identification accuracy from 70% to 95% and time-sensitive constraint detection accuracy from 59% to 94%, as evaluated against the ground-truth metadata of the intent dataset. This improvement reduces the dependence on manually specified keyword-to-profile mappings and enables more flexible interpretation of diverse natural-language intents, as discussed in Section II.

2) *Sub-Intent Generation Results:* Fig. 4 shows that all LMs achieve strong performance across semantic, lexical, and deployment-readiness metrics, with SBERT scores generally above 0.93 and FEACI exceeding 0.80. Claude Sonnet-4.6 consistently performs best, achieving the highest scores across all metrics. Among open-source models, Hermes-2-Pro and Phi-4-mini provide the strongest overall results, indicating that the proposed framework enables even compact models to generate high-quality sub-intents.

The impact of RAG augmentation is generally modest but positive. For models such as Sonnet-4.6, Hermes-2-Pro, and Phi-4-mini, RAG consistently improves or preserves semantic fidelity and deployment-readiness scores. Gains are most evident in ROUGE-L and Token-F1, suggesting that retrieved examples better align generated sub-intents with the expected structure. Minor degradations observed for some models are mainly caused by over-specialization toward retrieved examples, resulting in verbose or merged sub-intent formulations. However, these effects remain limited, and the overall trends confirm that RAG improves consistency without compromising semantic correctness.

Metric	Formula	What it evaluates
FEACI Format (F) ↑	$S_F = \frac{1}{G} \sum_{j=1}^G \mathbf{1}[g_j \text{ valid structurally}]$	Fraction of correct format: expected syntax in sub-intents and deployable <code>tc</code> rules.
FEACI Explainability (E) ↑	$S_E = \frac{1}{G} \sum_{j=1}^G \mathbf{1}[\exists k \in K : k \subset g_j]$	Fraction of items with reasoning keywords or explanation comments, K = set of explanation keywords.
FEACI Accuracy (A) ↑	$S_A = \frac{\sum_{m=1}^V w_m [v_m^{gold} \subset g_j]}{\sum_{m=1}^V w_m}$	Weighted fraction of outputs with correct configuration values (e.g., traffic-profile match assigned higher weight).
FEACI Cost (C_n) ↓	$C = c_i(N_{in}) + c_o(N_{out})$ $S_C = \begin{cases} C/C_0 & \text{if } C \leq 10C_0, \\ 1 & \text{otherwise} \end{cases}$	Normalized token cost (0 for open-source models; reference $C_0 = 0.1$ United States dollars (USD)).
FEACI Inference Time (I_n) ↓	$S_I = \min(I/I_0, 1)$	Normalized inference latency (threshold $I_0 = 60$ s).
Overall FEACI [6] ↑	$FEACI = \sum_{i=F,E,A,C,I} w_i S_i$	Weighted FEACI composite score ($w_i = 0.20$).

TABLE 1

AUTOMATED FEACI METRICS ADAPTED HERE FOR DEPLOYMENT READINESS. **LEGEND:** ↑/↓: HIGHER/LOWER IS BETTER; G/g_j : GENERATED OUTPUTS/INDIVIDUAL OUTPUT (SUB-INTENTS OR `tc` RULES); V : DATASET REFERENCE VALUES; S_C/S_I : NORMALIZED COST/INFERENCE SCORES. ESTABLISHED METRICS: [8], [18].

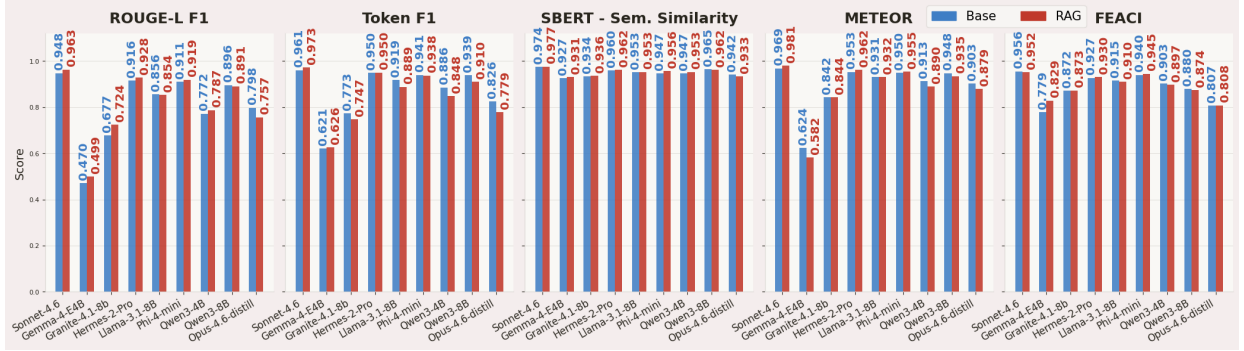
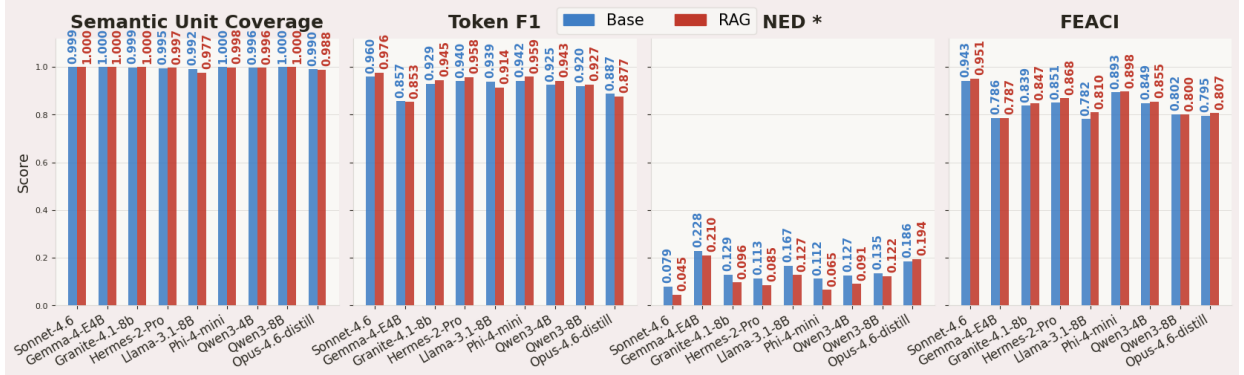


Fig. 4. Evaluated LMs: five-run mean intent-to-sub-intent performance.

Fig. 5. LMs: five-run mean sub-intent-to-`tc` performance; *NED**: lower is better

3) *Analysis of Generated Linux `tc` Rules:* Fig. 5 presents the Phase 2-C `tc` rule-generation results. Claude Sonnet-4.6 achieves the strongest overall performance, although SLMs such as Phi-4-mini remain highly competitive when provided with corrected sub-intents. Compared with the previous phase, performance differences across models are less pronounced, with most models achieving near-perfect semantic unit coverage and high Token-F1 scores. This reflects the deterministic nature of `tc` rules, which primarily require mapping semantic-model parameters into a rigid configuration syntax.

Consequently, smaller models can closely match larger LLMs. As in the previous phase, RAG-based retrieval generally improves `tc` generation. For Sonnet-4.6, NED is nearly halved, while semantic unit coverage, Token-F1, and FEACI improve. Similar trends are observed for most models, suggesting that retrieved examples help align generated configurations with the expected rule structure. Nevertheless, even the static few-shot setup achieves strong results, highlighting the effectiveness of the critique module in producing deployment-ready `tc` configurations.

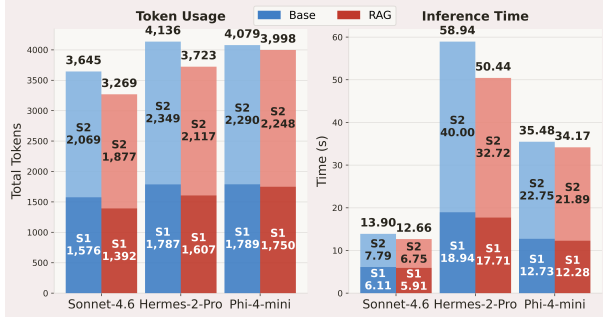


Fig. 6. Top models under Base/RAG: token use and inference time; S1/S2: intent-to-sub-intent/sub-intent-to-`tc` generation.

4) *Token Usage and Inference Time Analysis*: Fig. 6 compares the token usage and inference time of the best-performing models under Base and RAG settings. Overall, RAG reduces token consumption and latency while maintaining comparable semantic fidelity, structural precision, and deployment readiness. The gains are most evident for Claude Sonnet-4.6, where total simulation cost decreases from approximately \$1.87 to \$1.72, with average reductions of ~ 375 tokens and ~ 1.25 seconds per intent. Hermes-2-Pro and Phi-4-mini show similar efficiency gains, indicating that retrieved examples improve generation efficiency without sacrificing output quality.

We compare our framework with the single-pass pipeline of [8] using Claude Sonnet-4.6, the best-performing backbone LLM in our evaluation. We observe improvements in both translation stages: in Phase 2-B, dynamic metadata extraction increases ROUGE-L, Token-F1, and SBERT by 0.07, 0.06, and 0.06, respectively; in Phase 2-C, critique-driven correction improves Token-F1 by 0.065 and reduces NED by 0.218. The proposed framework also lowers per-intent token usage by approximately 23,500 tokens, reduces inference latency by 2.2 seconds, and achieves an overall cost reduction of approximately \$6 on the full dataset.

V. CONCLUSION

This work presents a closed-loop framework for translating traffic-shaping intents into deployable Linux `tc` configurations through the integration of DT semantic modeling, critique-driven correction, and RAG-based knowledge reuse. The reported experimental results (average values across five independent runs) demonstrate high translation quality and robustness, achieving strong semantic fidelity, configuration accuracy, and deployment readiness. RAG augmentation further improves efficiency by reducing token consumption and inference latency while maintaining or improving translation quality. Notably, compact models such as Phi-4-mini achieve performance comparable to that of larger LLMs in both sub-intent and `tc` generation, while having an approximately 50% smaller model size, lower random-access memory (RAM) and video random-access memory (VRAM) usage, and 33% lower inference time, as shown in Figs. 4–6. Future work includes dynamic telemetry-driven semantic modeling, persistent multi-run RAG knowledge bases, larger-scale real-world evaluations, and CM extensions for edge-case rule conflicts.

ACKNOWLEDGMENT

This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) under the DISCOVERY and CREATE TRAVERSAL Programs.

REFERENCES

- [1] A. Leivadreas and M. Falkner, “A survey on intent-based networking,” *IEEE Communications Surv. & Tut.*, vol. 25, no. 1, pp. 625–655, 2022.
- [2] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, “Intent-Based Networking—concepts and definitions,” Internet Engineering Task Force, Request for Comments 9315, October 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9315>
- [3] L. Velasco, M. Signorelli, O. G. De Dios, C. Papagianni, R. Bifulco, J. J. V. Olmos, S. Pryor, G. Carrozzo, J. Schulz-Zander, M. Bennis *et al.*, “End-to-end intent-based networking,” *IEEE Communications Magazine*, vol. 59, no. 10, pp. 106–112, 2021.
- [4] H. Yu, H. Rahimi, C. Janz, D. Wang, C. Yang, and Y. Zhao, “A comprehensive framework for intent-based networking, standards-based and open-source,” in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2023, pp. 1–6.
- [5] K. Dzeparowska, J. Lin, A. Tizghadam, and A. Leon-Garcia, “LLM-based policy generation for intent-based management of applications,” in *2023 19th International Conference on Network and Service Management (CNSM)*. IEEE, 2023, pp. 1–7.
- [6] L. Dinh, S. Cherrared, X. Huang, and F. Guillemin, “Towards end-to-end network intent management with large language models,” *arXiv preprint arXiv:2504.13589*, 2025.
- [7] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, “NetConfEval: Can LLMs facilitate network configuration?” *Proc. ACM on Networking*, vol. 2, no. CoNEXT2, pp. 1–25, 2024.
- [8] S. Acharya and B. Kantarci, “Intent2QoS: Language model-driven automation of traffic shaping configurations,” in *ICC 2026-IEEE International Conference on Communications*. IEEE, 2026, pp. 1–6.
- [9] J. Li, S. Zou, Y. Sun, H. Gao, and W. Ni, “Business intent and network slicing correlation dataset from data-driven perspective,” *Scientific Data*, vol. 12, no. 1, p. 419, 2025.
- [10] HuggingFace, “Hugging Face Hub,” accessed: Jun. 2, 2026. [Online]. Available: <https://huggingface.co>
- [11] C. Pfefferle, F. Wiedner, and C. Schwarzenberg, “IEEE 802.1Qcr asynchronous traffic shaping with Linux traffic control,” *Network*, vol. 11, 2021.
- [12] F. A. Bimo, M. A. C. Galdon, C.-K. Lai, R.-G. Cheng, and E. K. Chong, “Intent-based network for RAN management with large language models,” *arXiv preprint arXiv:2507.14230*, 2025.
- [13] Y. Wei, X. Xie, T. Hu, Y. Zuo, X. Chen, K. Chi, and Y. Cui, “INTA: Intent-based translation for network configuration with LLM agents,” in *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*. IEEE, 2025, pp. 1–16.
- [14] M. K. Hossain and W. Aljoby, “NetIntent: Leveraging large language models for end-to-end intent-based SDN automation,” *IEEE Open Journal of the Communications Society*, vol. 6, pp. 10 512–10 541, 2025.
- [15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proc. Conf. on Empirical Methods in Natural Lang. Processing and the Intl. Joint Conf. on Natural Lang. Processing*, 2019, pp. 3982–3992.
- [16] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text summarization branches out*, 2004, pp. 74–81.
- [17] H. Schütze, C. D. Manning, and P. Raghavan, *Introduction to information retrieval*. Cambridge University Press Cambridge, 2008, vol. 39.
- [18] M. Denkowski and A. Lavie, “METEOR universal: Language specific translation evaluation for any target language,” in *Proceedings of the ninth workshop on statistical machine translation*, 2014, pp. 376–380.
- [19] B. Peng, C. Zhu, C. Li, X. Li, J. Li, M. Zeng, and J. Gao, “Few-shot natural language generation for task-oriented dialog,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 172–182.
- [20] L. Yujian and L. Bo, “A normalized Levenshtein distance metric,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 6, pp. 1091–1095, 2007.